

Detecção Online em Séries Temporais: Streaming, Drift e Latência

O cenário online: restrições e objetivos

Por que o cenário muda o problema

No offline, a série completa está disponível. No online, observações chegam continuamente e a decisão precisa ser tomada no fluxo, sem acesso ao futuro.

Detecção online

- Observações chegam em sequência.
- O detector decide usando apenas o que já chegou: $\delta_t = D(x_1, \dots, x_t)$.
- A decisão deve ser rápida o suficiente para permitir reação operacional.

Predição online

- O objetivo é prever a ocorrência futura de um evento.
- É uma forma antecipatória de event detection.
- A utilidade depende de emitir alerta antes que a ação deixe de ser possível.

Equilíbrio necessário

O método precisa equilibrar rapidez, precisão, memória e capacidade de adaptação ao longo do fluxo.

Três cenários de eventos

Os três cenários

Offline detection

Reconhecer eventos passados. A série completa está disponível para análise.

Online detection

Detectar eventos enquanto eles acontecem. Decisão tomada no fluxo.

Online prediction

Antecipar eventos antes que ocorram. Alerta emitido com horizonte h .

Detecção offline – características



A série já foi coletada antes da análise.



O algoritmo pode olhar para passado e futuro em torno de um candidato a evento.



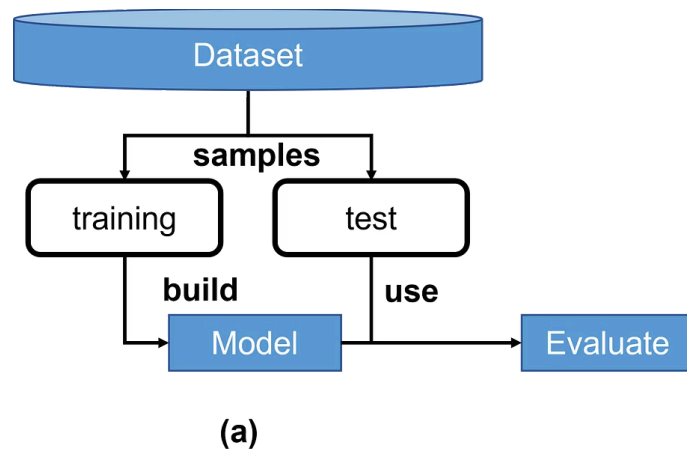
É adequada para auditoria, explicação retrospectiva e benchmark de métodos.

O cenário offline serve de referência para entender o que se perde — e o que se ganha — ao operar online.

Treino e teste: tradicional vs streaming

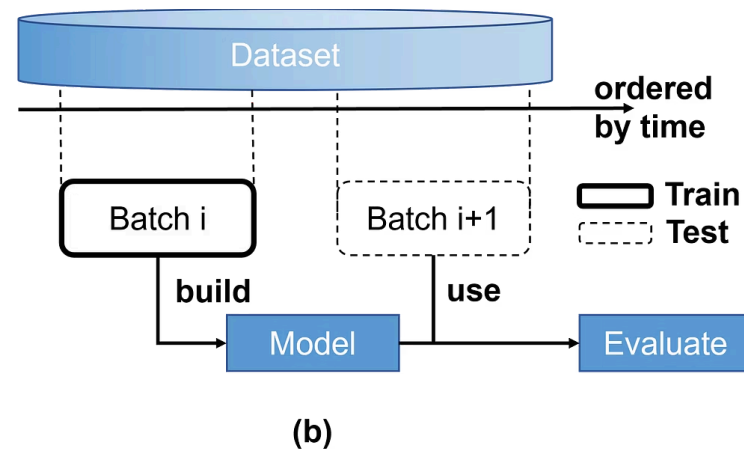
Cenário tradicional

- A série é dividida em treino e teste de forma estática.
- O modelo é ajustado com dados passados e avaliado em dados mais recentes.
- Validação temporal e ajuste de hiperparâmetros ocorrem antes da implantação.



Cenário streaming

- Cada batch pode treinar ou atualizar o modelo para o próximo batch.
- A ordem temporal é parte do protocolo experimental.
- Esse desenho imita a operação de um sistema que aprende com o fluxo.



No streaming, o batch i é usado para construir ou atualizar o modelo avaliado no batch $i+1$.

O que é online detection

Definição operacional

É a identificação de eventos conforme as observações chegam, sem acesso ao futuro. O detector processa a série sequencialmente e emite δ_t a cada passo.

Condição de causalidade

$\delta_t = D(x_1, \dots, x_t)$. Nenhuma observação posterior a t pode ser usada na decisão em t .

Restrição de latência

A decisão deve ser emitida dentro de um orçamento de tempo Δt compatível com a taxa de chegada do fluxo.

Saída do detector

$\delta_t \in \{0, 1\}$: 0 indica comportamento normal, 1 indica evento detectado. Alguns detectores emitem também um escore de anomalia $s_t \in \mathbb{R}$.

Loop geral de detecção online

Estrutura do loop

O detector opera em um loop potencialmente infinito. A cada passo t , recebe x_t , atualiza seu estado interno e emite δ_t .

$$\delta_t = D(x_t; \theta_t)$$

Parâmetros: θ_t (estado interno do detector em t), τ (limiar de decisão), s_t (escore de anomalia).

Condições de parada

O loop termina quando o fluxo é encerrado ou quando uma condição externa interrompe o monitoramento.

Pseudocódigo – Loop geral de detecção online

ENTRADA: fluxo $X = x_1, x_2, \dots$, detector D , limiar τ

PRÉ-CONDIÇÃO: D inicializado com estado θ_0

PASSO 1 — Ao receber x_t do fluxo:

se fluxo encerrado $\rightarrow$ parar

PASSO 2 — Calcular escore de anomalia:

$s_t \leftarrow D.\text{escore}(x_t; \theta_t)$

PASSO 3 — Tomar decisão:

$\delta_t \leftarrow 1$ se $s_t > \tau$, senão 0

PASSO 4 — Atualizar estado interno:

$\theta_{t+1} \leftarrow D.\text{atualizar}(\theta_t, x_t)$

PASSO 5 — Emitir δ_t (e opcionalmente s_t)

PASSO 6 — $t \leftarrow t + 1$; voltar ao PASSO 1

SAÍDA: sequência de decisões $\delta_1, \delta_2, \dots$

PÓS-CONDIÇÃO: cada δ_t usa apenas $x_1, \dots, x_t$ (causalidade garantida)

Formalização do fluxo temporal

Definição formal

Seja $X = \langle x_1, x_2, \dots, x_t, \dots \rangle$ um fluxo potencialmente infinito de observações. Em cada instante t , o detector D recebe x_t e emite uma decisão $\delta_t \in \{0, 1\}$ usando apenas o histórico disponível.

$$\delta_t = D(x_1, x_2, \dots, x_t)$$

Parâmetros: $x_t \in \mathbb{R}^d$ (observação), δ_t (decisão binária), t (instante corrente).

Restrição causal

O detector não pode acessar $x_{t+1}, x_{t+2}, \dots$ no momento da decisão. Isso distingue o cenário online do offline, onde toda a série está disponível.

$$\langle \dots, x_{t-2}, x_{t-1}, x_t \mid x_{t+1}, x_{t+2}, \dots \rangle$$

A barra vertical indica o limite de visibilidade em t .

Tensão inevitável

Alguns eventos só ficam claros depois de algumas observações. Usar observações futuras viola o cenário operacional. Isso cria uma tensão entre decisão imediata e confirmação confiável.

Early detection e o trade-off rapidez vs falso positivo

Definição

Detectar o evento o mais cedo possível após seu início, minimizando a latência $L = \hat{t} - t^*$.

minimizar $E[L]$ sujeito a $FPR \leq \alpha$

Parâmetros: L (latência), FPR (taxa de falso positivo), α (nível de tolerância a alarmes falsos).

Valor operacional

O valor do alerta decai com o tempo: $V(\text{alerta}) = f(h - L)$, onde h é o horizonte de ação. Em sistemas críticos, atraso pequeno pode ser irreversível.

O trade-off central

Detector sensível

Reage rapidamente. Pode antecipar eventos importantes. Tende a gerar mais falsos positivos e fadiga de alerta.

Detector conservador

Exige mais evidência. Reduz alarmes indevidos. Pode detectar tarde ou perder eventos em sistemas críticos.

Em muitos domínios, o custo de um falso negativo supera o de um falso positivo — o que justifica limiares mais baixos e detecção antecipada.

Fast data e limitações do offline

Fast data e sistemas críticos

Fast data envolve grande volume, alta velocidade e mudança em intervalos curtos.

- Redes, IoT, indústria, saúde e segurança exigem monitoramento contínuo.
- A solução precisa operar sob restrições de tempo, memória e confiabilidade.
- O custo de uma detecção tardia pode ser irreversível em sistemas críticos.

Por que offline não migra diretamente

- Métodos offline podem depender da série completa.
- O custo computacional pode ser incompatível com fluxo contínuo.
- A presença de drift muda a noção de comportamento típico.
- Métricas offline podem ignorar o atraso temporal da detecção.

Modelos estáticos vs dinâmicos e estratégias de memória

Estáticos

- Treinados antes da implantação.
- Permanecem inalterados durante a execução.
- Funcionam quando a distribuição é estável.

Full-memory

Mantém todas as observações vistas. Preserva histórico completo, mas tende a ser impraticável para fluxos longos ou de alta dimensão.

Dinâmicos

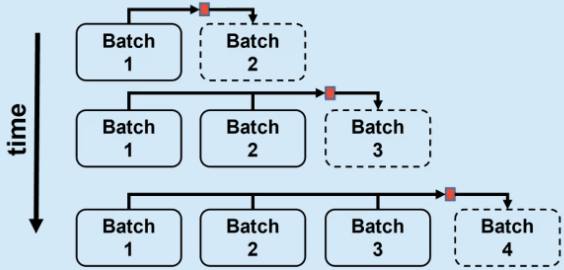
- São atualizados com novos dados.
- Podem acompanhar mudanças no fluxo.
- Exigem política de memória e adaptação.

No-memory

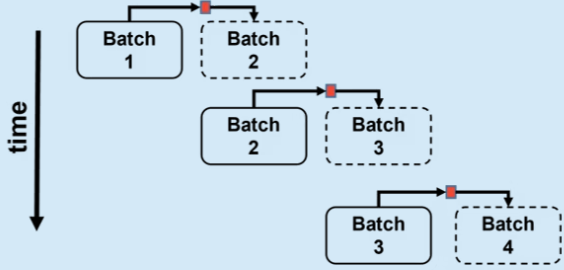
Atualizado incrementalmente sem armazenar histórico bruto. Leve em memória, mas pode esquecer padrões úteis ou sofrer com atualizações ruidosas.

A política de memória define o que o modelo lembra e o que esquece — e determina diretamente sua capacidade de adaptação.

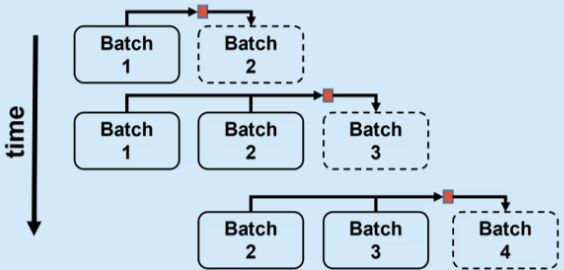
(a) full memory



(b) no memory



(c) windowed memory



Windowed memory

Definição

Mantém apenas uma janela W de tamanho w das observações ou batches mais recentes.

$$W_t = \{x_{t-W+1}, \dots, x_t\}$$

Parâmetros: w (tamanho da janela), x_t (observação atual).

Trade-off

w grande → mais contexto, mais memória. w pequeno → adaptação rápida, menos histórico. A escolha de w controla o quanto o modelo lembra do passado.

Pseudocódigo – Gerenciamento por janela

ENTRADA: fluxo $X = x_1, x_2, \dots$, tamanho de janela w
PRÉ-CONDIÇÃO: $w \geq 1$; buffer inicializado com histórico disponível

- PASSO 1 — Ao receber x_t :
 $W_{\{t-1\}} \leftarrow \text{buffer}$
- PASSO 2 — Treinar ou atualizar modelo M usando $W_{\{t-1\}}$
- PASSO 3 — Detectar o ponto atual:
 $\delta_t \leftarrow M.\text{detectar}(x_t)$
- PASSO 4 — Atualizar memória:
 $\text{buffer.append}(x_t)$
 se $\text{len}(\text{buffer}) > w$:
 $\text{buffer.remove_first}()$
- PASSO 5 — Emitir δ_t

SAÍDA: sequência de decisões $\delta_1, \delta_2, \dots$
PÓS-CONDIÇÃO: δ_t usa apenas informação disponível antes ou no instante t , sem treinar no próprio ponto antes da decisão

Full-memory, no-memory e windowed memory definem estratégias diferentes de retenção do histórico.

Retreinamento por batches recentes

Definição

O modelo é reconstruído periodicamente usando apenas os B batches mais recentes, descartando dados antigos.

$$M_{t+1} \leftarrow \text{treinar}(\{B_{t-B+1}, \dots, B_t\})$$

Parâmetros: B (número de batches retidos), B_t (batch atual), período de retreinamento r .

Quando usar

Útil quando o drift é abrupto e o modelo antigo se torna rapidamente obsoleto. Custo computacional maior que aprendizado incremental.

Pseudocódigo – Retreinamento por batches recentes

ENTRADA: fluxo em batches $B_1, B_2, \dots$, janela B , período r

PRÉ-CONDIÇÃO: M_0 treinado, fila de batches inicializada vazia

PASSO 1 — Ao receber batch B_t :

`fila.append( $B_t$ )`

se `len(fila) >  $B$` :

`fila.remove_first()`

PASSO 2 — Se $t \bmod r = 0$ (período de retreinamento):

$M_{t+1} \leftarrow \text{treinar}(\text{fila})$

Senão:

$M_{t+1} \leftarrow M_t$

PASSO 3 — Para cada $x_t \in B_t$:

$\delta_t \leftarrow M_{t+1}.\text{detectar}(x_t)$

emitir δ_t

PASSO 4 — $t \leftarrow t + 1$; voltar ao PASSO 1

SAÍDA: sequência $\delta_1, \delta_2, \dots$ e modelo retreinado periodicamente

PÓS-CONDIÇÃO: modelo usa apenas os B batches mais recentes

Aprendizado incremental

Definição

O modelo M_t é atualizado a cada novo exemplo ou batch, produzindo M_{t+1} sem rever o histórico completo.

$$M_{t+1} \leftarrow \text{update}(M_t, x_t)$$

Parâmetros: M_t (modelo atual), x_t (nova observação), η (taxa de aprendizado).

Desafios

Ruído pode degradar o modelo a cada passo. Drift real deve ser absorvido, mas ruído não.

Esquecimento catastrófico pode apagar padrões úteis.

Pseudocódigo – Aprendizado incremental

ENTRADA: fluxo $X = x_1, x_2, \dots$, modelo inicial M_0 , taxa η

PRÉ-CONDIÇÃO: M_0 treinado em dados históricos ou aleatório

PASSO 1 — Ao receber x_t :

$$\delta_t \leftarrow M_t.\text{detectar}(x_t)$$

PASSO 2 — Calcular gradiente ou estatística de atualização:

$$\Delta \leftarrow \text{gradiente}(M_t, x_t)$$

PASSO 3 — Atualizar modelo:

$$M_{t+1} \leftarrow M_t - \eta \cdot \Delta$$

PASSO 4 — Emitir δ_t

PASSO 5 — $t \leftarrow t + 1$; $M_t \leftarrow M_{t+1}$; voltar ao PASSO 1

SAÍDA: sequência $\delta_1, \delta_2, \dots$ e modelo atualizado M_t

PÓS-CONDIÇÃO: M_t reflete os dados mais recentes sem rever histórico

Não estacionaridade e concept drift

Não estacionaridade

O processo gerador pode mudar ao longo do tempo. O que era típico ontem pode deixar de ser típico hoje. Essa mudança afeta diretamente detectores treinados no passado.

Concept drift – definição formal

Concept drift ocorre quando a distribuição conjunta $P(X, y)$ muda ao longo do tempo no fluxo.

$$\exists t_1 < t_2 : P_{\{t_1\}}(X, y) \neq P_{\{t_2\}}(X, y)$$

Aproxima detecção online de change point detection e adaptação de modelo.

Tipos de drift

Abrupto

P muda instantaneamente em t^* .
Requer retreinamento imediato.

Gradual

P transiciona suavemente.
Aprendizado incremental com λ pequeno é adequado.

Recorrente

P retorna a estado anterior. Modelos com memória de conceitos anteriores são vantajosos.

Dilema estabilidade-plasticidade

Definição

Todo sistema online enfrenta a tensão entre resistir a variações irrelevantes (estabilidade) e adaptar-se rapidamente a mudanças reais (plasticidade).

equilíbrio: $\alpha \cdot \text{estabilidade} + (1-\alpha) \cdot \text{plasticidade}$

Parâmetro $\alpha \in [0,1]$ controla o balanço. $\alpha \rightarrow 1$: sistema rígido. $\alpha \rightarrow 0$: sistema altamente adaptativo.

Consequências do desequilíbrio

Excesso de estabilidade: drift não é absorvido, modelo envelhece. Excesso de plasticidade: ruído e eventos raros corrompem o modelo.

Manifestações práticas

Taxa de aprendizado η

η alto $\rightarrow$ plasticidade. η baixo $\rightarrow$ estabilidade. Schedules adaptativos ajustam η dinamicamente.

Limiar de drift α

α baixo $\rightarrow$ detecta drift facilmente (mais plástico). α alto $\rightarrow$ exige evidência forte (mais estável).

Tamanho da janela w

w pequeno $\rightarrow$ adaptação rápida. w grande $\rightarrow$ memória longa e resistência a ruído.

Regularização

Penalizar mudanças grandes no modelo favorece estabilidade sem sacrificar toda a plasticidade.

Active learning para drift

Definição

O sistema detecta drift antes de adaptar o modelo. A atualização só ocorre quando há evidência estatística de mudança no conceito.

se $\text{drift_detectado}(x_t) \rightarrow M_{t+1} \leftarrow \text{retreinar}(M_t, W_t)$

Parâmetros: detector de drift D_{drift} , limiar α , janela de referência W_{ref} .

Vantagem e risco

Reduz adaptações desnecessárias causadas por ruído. Depende da qualidade do detector de drift: falsos negativos atrasam a adaptação.

Pseudocódigo – Adaptação ativa a drift

ENTRADA: fluxo X , modelo M_0 , detector de drift D_{drift} , limiar α

PRÉ-CONDIÇÃO: M_0 treinado, W_{ref} inicializada com dados históricos

PASSO 1 — Ao receber x_t :

$\delta_t \leftarrow M_t.\text{detectar}(x_t)$

PASSO 2 — Testar drift:

$p_{\text{valor}} \leftarrow D_{\text{drift}}.\text{testar}(W_{\text{ref}}, W_t)$

$\text{drift} \leftarrow (p_{\text{valor}} < \alpha)$

PASSO 3 — Se drift = verdadeiro:

$M_{t+1} \leftarrow \text{retreinar}(M_t, W_t)$

$W_{\text{ref}} \leftarrow W_t$

Senão:

$M_{t+1} \leftarrow M_t$ (sem alteração)

PASSO 4 — Emitir δ_t

PASSO 5 — $t \leftarrow t + 1$; voltar ao PASSO 1

SAÍDA: sequência $\delta_1, \delta_2, \dots$ e modelo adaptado

PÓS-CONDIÇÃO: modelo só muda quando drift é confirmado

Passive learning para drift

Definição

O modelo é atualizado continuamente a cada batch ou observação, independentemente de alarme de drift.

$$M_{t+1} \leftarrow \text{update}(M_t, x_t) \quad \forall t$$

Parâmetros: taxa de esquecimento $\lambda \in (0,1]$, peso de observações por idade. λ próximo de 1 preserva memória longa. λ menor acelera o esquecimento. O peso decai com a idade da observação, não com o tempo absoluto da observação corrente.

Vantagem e risco

Acompanha mudanças graduais sem necessidade de detector de drift. Pode absorver ruído ou eventos raros como comportamento normal, degradando a especificidade.

Pseudocódigo – Adaptação passiva a drift

ENTRADA: fluxo X , modelo M_0 , taxa de esquecimento λ

PRÉ-CONDIÇÃO: M_0 treinado, $\lambda \in (0,1]$

PASSO 1 — Ao receber x_t :

$$\delta_t \leftarrow M_t.\text{detectar}(x_t)$$

PASSO 2 — Atribuir pesos por idade:

para cada observação antiga x_i no histórico:

$$\text{peso}_i \leftarrow \lambda^{(t-i)}$$

a nova observação x_t entra com peso 1

PASSO 3 — Atualizar modelo:

$$M_{t+1} \leftarrow \text{update}(M_t, x_t, \text{peso}=1, \text{histórico ponderado por peso}_i)$$

PASSO 4 — Emitir δ_t

PASSO 5 — $t \leftarrow t + 1$; voltar ao PASSO 1

SAÍDA: sequência $\delta_1, \delta_2, \dots$ e modelo continuamente atualizado

PÓS-CONDIÇÃO: observações mais antigas têm peso $\lambda^{(t-i)} \rightarrow 0$, garantindo esquecimento gradual por idade

Online prediction

Definição

Vai além de detectar eventos passados ou em andamento. Busca prever se um evento ocorrerá em uma janela futura $[t+1, t+h]$.

$$P(\text{evento em } [t+1, t+h] \mid x_1, \dots, x_t)$$

Parâmetros: h (horizonte de previsão), limiar de probabilidade p^* para emissão de alerta.

Valor operacional

Útil quando o objetivo é mitigação, prevenção ou alerta antecipado. O valor do alerta depende do tempo disponível para agir antes do evento.

Abordagens

Por classificação

Treina classificador com rótulos de evento. Requer instâncias rotuladas e lida com desbalanceamento.

Por regressão

Prevê valores futuros da série e aplica detector sobre a previsão. Dispensa rótulos diretos.

Por probabilidade

Estima $P(\text{evento} \mid \text{histórico})$ diretamente. Permite calibrar o limiar de alerta por custo-benefício.

Predição por classificação

Definição

Extraí sequências e atributos de observações anteriores e treina um classificador para prever rótulos de evento.

$$\hat{y}_{t+1} = C(\varphi(x_1, \dots, x_t))$$

Parâmetros: C (classificador), φ (função de extração de atributos), $\hat{y} \in \{0,1\}$ (rótulo previsto).

Requisitos

Exige rótulos confiáveis para treinar o classificador. Adequada quando eventos são bem definidos e rotulação é viável.

Desafios operacionais

1

Instâncias rotuladas escassas

Muitos fluxos têm poucas instâncias rotuladas disponíveis.

2

Forte desbalanceamento

Eventos são raros: razão positivo/negativo pode ser 1:1000 ou pior.

3

Rotulação cara e imprecisa

Obter rótulos por especialistas pode ser lento, caro ou sujeito a erro.

Predição por regressão

Definição

Usa o histórico $X = \langle x_1, \dots, x_t \rangle$ para prever valores futuros da série. A detecção de eventos é aplicada sobre a série prevista.

$$\hat{x}_{t+h} = F(x_1, \dots, x_t)$$

Parâmetros: F (modelo de regressão), h (horizonte de previsão), ϵ (erro de previsão).

Vantagem

Funciona sem rótulos diretos de evento. O evento é inferido quando $\hat{x}_{t+h}$ ultrapassa um limiar ou exhibe padrão anômalo.

Fluxo de decisão

Receber x_t e atualizar histórico H .

Prever: $\hat{x}_{t+1:t+h} \leftarrow F(H)$.

Aplicar detector D sobre $\hat{x}_{t+1:t+h}$.

Emitir alerta se D detectar evento na série prevista.

Detectar eventos na série prevista

Abordagem em dois estágios

Primeiro prevê-se uma sequência futura $\hat{X}_{t+1:t+h}$ usando o histórico disponível. Depois aplica-se um detector sobre a série prevista.

$$\hat{X}_{t+1:t+h} = F(x_1, \dots, x_t)$$

Parâmetros: F (modelo de previsão), h (horizonte), D (detector de eventos).

Interpretação

$\hat{\delta}_{t+k} = D(\hat{X}_{t+1:t+k})$: evento previsto no horizonte k. Permite emitir alerta antes da ocorrência real.

Pseudocódigo – Predição online seguida de detecção

ENTRADA: fluxo X, modelo de previsão F, detector D, horizonte h
PRÉ-CONDIÇÃO: F treinado, D calibrado, $h \geq 1$

PASSO 1 — Ao receber x_t :

atualizar histórico: $H \leftarrow H \cup \{x_t\}$

PASSO 2 — Prever sequência futura:

$$\hat{X}_{t+1:t+h} \leftarrow F(H)$$

PASSO 3 — Aplicar detector sobre série prevista:

para $k = 1$ até h:

$$\hat{\delta}_{t+k} \leftarrow D(\hat{X}_{t+1:t+k})$$

PASSO 4 — Se $\exists k : \hat{\delta}_{t+k} = 1$:

emitir alerta antecipado com horizonte k

PASSO 5 — $t \leftarrow t + 1$; voltar ao PASSO 1

SAÍDA: alertas antecipados com horizonte estimado k

PÓS-CONDIÇÃO: alerta emitido antes do evento real em $t+k$

Online multivariate anomaly detection

Definição

Detecta anomalias em fluxos com múltiplas variáveis correlacionadas, onde a anomalia pode estar nas relações entre dimensões, não apenas nos valores individuais.

$$s_t = \text{anomalia}(x_t \in \mathbb{R}^d, W_t)$$

Parâmetros: d (dimensionalidade), W_t (janela de referência multivariada).

Desafios específicos

Alta dimensionalidade e relações que mudam no tempo. Anomalias podem ser invisíveis em cada dimensão isolada, mas evidentes na estrutura conjunta.

Abordagens

Distance-based

COD, MCOD: distância no espaço $\mathbb{R}^d$. Sofre com maldição da dimensionalidade.

Tree-based

HS-Tree, iForestASD: particionamento do espaço multivariado. Eficiente e incremental.

Projection-based

LODA, RS-Hash: projeção para $\mathbb{R}^m$ com $m \ll d$. Escalável para alta dimensão.

Métodos distance-based

Princípio

Monitoram distâncias entre observações no fluxo. Uma observação é anômala se está distante de seus vizinhos mais próximos.

$$s_t = \text{dist}(x_t, \text{kNN}(x_t, W_t))$$

Parâmetros: k (número de vizinhos), W_t (janela de referência), dist (métrica de distância).

Algoritmos

COD (Continuous Outlier Detection): acompanha outliers baseados em distância continuamente. MCOD: versão com manutenção eficiente da janela deslizando.

Propriedades operacionais

Sem suposição de distribuição

Não assume forma paramétrica para $P(X)$. Robusto a distribuições arbitrárias.

Sensível à dimensionalidade

Em alta dimensão, distâncias se concentram. Projeções ou redução de dimensionalidade podem ser necessárias.

Custo por observação

$O(k \cdot |W_t|)$ por passo. Estruturas de índice (KD-tree, LSH) reduzem o custo em alta dimensão.

Métodos tree-based

Princípio

Particionam o espaço de dados recursivamente para isolar observações anômalas. Anomalias são isoladas com menos partições.

$$s_t = \text{profundidade_de_isolamento}(x_t, T)$$

Parâmetros: T (estrutura de árvore), profundidade esperada de isolamento, janela de atualização.

Algoritmos

HS-Tree (Half-Space Tree): atualiza uma estrutura hierárquica de semi-espacos a cada batch. iForestASD: Isolation Forest adaptado para fluxos com drift.

Propriedades operacionais

Eficiência

Custo de detecção $O(\log n)$ por observação após construção da árvore.

Atualização incremental

HS-Tree atualiza contadores internos sem reconstruir a árvore a cada passo.

Interpretabilidade

O caminho de isolamento pode indicar quais dimensões contribuem para a anomalia.

Métodos projection-based

Princípio

Reduzem a dimensionalidade por projeções aleatórias ou aprendidas, detectando anomalias no espaço projetado.

$$\tilde{x}_t = Px_t, \quad s_t = \text{anomalia}(\tilde{x}_t)$$

Parâmetros: $P \in \mathbb{R}^{m \times d}$ (matriz de projeção), $m \ll d$ (dimensão reduzida).

Algoritmos

LODA (Lightweight Online Detector of Anomalies): usa histogramas em projeções aleatórias 1D. RS-Hash: combina projeções com hashing para fluxos de alta dimensão.

Propriedades operacionais

Escalabilidade

Custo $O(m \cdot d)$ por observação. Adequado para fluxos de alta dimensão onde métodos distance-based falham.

Atualização leve

Histogramas e hashes são atualizados incrementalmente sem rever o histórico.

Perda de informação

A projeção pode descartar dimensões relevantes. Múltiplas projeções independentes compensam essa limitação.

Latência e viés temporal

Definição de latência

Seja t^* o instante real do evento e $\hat{t}$ o instante da primeira detecção. A latência de detecção é:

$$L = \hat{t} - t^*$$

$L > 0$: detecção tardia. $L < 0$: detecção antecipada (possível em predição). $L = 0$: detecção imediata.

Métricas derivadas

Mean Time To Detect (MTTD): média de L sobre todos os eventos. Alarmes persistentes: $\delta_t = 1$ por múltiplos passos após t^* .

Pseudocódigo – Cálculo de latência

ENTRADA: sequência de eventos reais $E = \{t^*_1, \dots, t^*_n\}$

sequência de detecções $D = \{\hat{t}_1, \dots, \hat{t}_m\}$

PRÉ-CONDIÇÃO: cada $\hat{t}_i$ é associado ao evento mais próximo em E

PASSO 1 — Para cada par $(t^*_i, \hat{t}_i)$:

$$L_i \leftarrow \hat{t}_i - t^*_i$$

PASSO 2 — Classificar:

se $L_i > 0 \rightarrow$ detecção tardia

se $L_i < 0 \rightarrow$ detecção antecipada

se $L_i = 0 \rightarrow$ detecção imediata

PASSO 3 — Calcular:

$$\text{MTTD} \leftarrow \text{média}(L_1, \dots, L_n)$$

alarmes_persistentes $\leftarrow$ contar runs de $\delta_t=1$ após $\hat{t}_i$

SAÍDA: vetor L , MTTD, contagem de alarmes persistentes

PÓS-CONDIÇÃO: L permite comparar detectores por viés temporal

Métodos híbridos

Motivação

Uma série pode conter simultaneamente anomalias pontuais, change points e motifs. Detectores especializados capturam evidências diferentes e complementares.

$$\delta_t = \text{combinar}(\delta_t^1, \delta_t^2, \dots, \delta_t^k)$$

Parâmetros: δ_t^i (decisão do detector i), regra de combinação (votação, média de escores, ensemble).

Estratégias de combinação

Votação majoritária: $\delta_t = 1$ se maioria dos detectores concorda. Média de escores: $s_t = (1/k)\sum s_t^i$, depois limiarizar. Stacking: meta-modelo aprende a combinar os escores.

Arquitetura híbrida

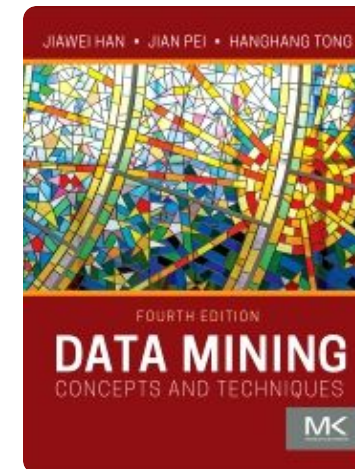
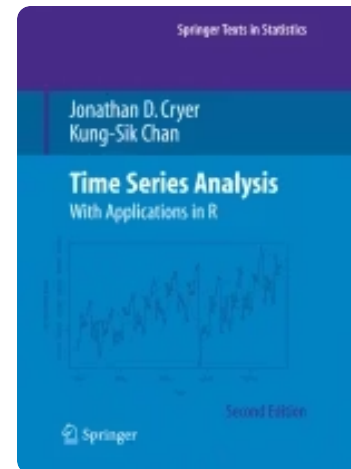
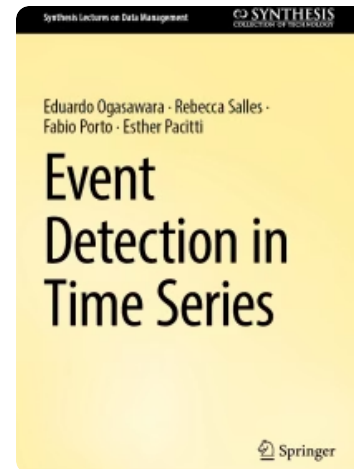
Receber x_t e distribuir para k detectores especializados $D^1, \dots, D^k$.

Cada D^i emite escore s_t^i e decisão δ_t^i de forma independente.

Módulo de combinação agrega $\{s_t^1, \dots, s_t^k\} \rightarrow s_t$ e aplica limiar $\tau \rightarrow \delta_t$.

Emitir δ_t com indicação de qual detector contribuiu para o alerta.

Referências Bibliográficas



Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.;
Pacitti, E. Springer Nature Switzerland,
2025.

Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.

Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan
Kaufmann, 4^a ed., 2022.